

DevOps Engineer 박상웅

Tech Blog <https://www.xxng1.com/>

Github <https://github.com/xxng1>



1. Well-Architected Framework에 대한 이해

- 전공, SW아카데미, Cloud 부트캠프를 거쳐서 Cloud Native 및 System Architecture에 대한 지식을 키워왔습니다.
- **운영 우수성, 보안, 신뢰성, 성능 효율성, 비용 최적화, 지속 가능성**에 대한 관점으로 서비스를 제공하기 위해 노력합니다.

2. DevOps 엔지니어링에 대한 마음가짐

- 언제든지 상황에 **신속히 대응**하여 서비스를 운영하기 위한 준비된 마음가짐과 책임감 있는 자세를 갖추고 있습니다.
- **반복되는 일을 자동화**하고 **비효율적인 프로세스를 개선**하는 것을 통해 더 나은 운영 환경을 만들고자 합니다.

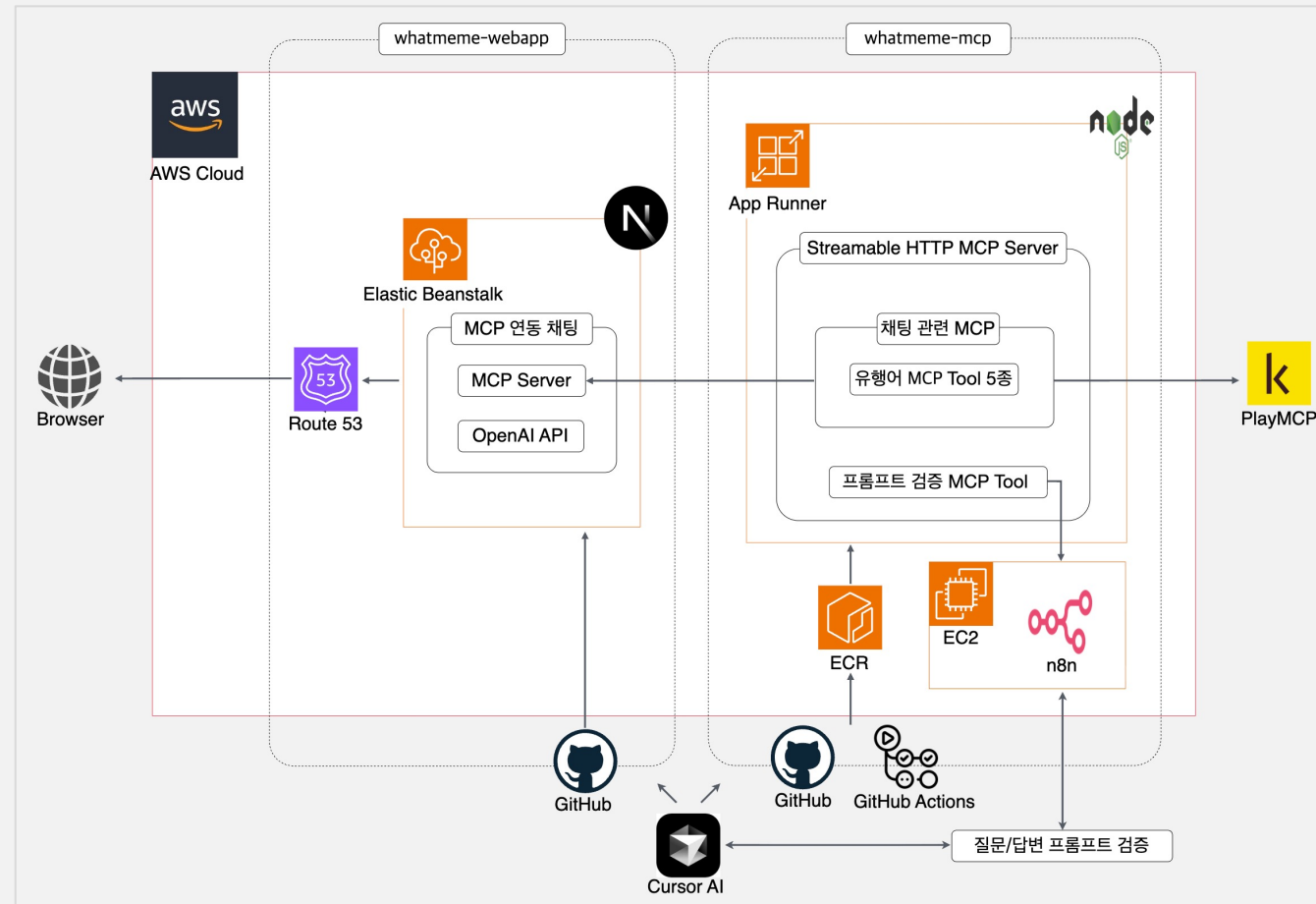
3. AI Natvie 엔지니어링에 대한 마음가짐

- 자동화 및 AI 도구들을 활용해 효율적으로 업무를 수행하는 능력을 키우고 있습니다.
- AI가 쏟아내는 산출물에 대한 **검증 부채**를 줄여 더욱 효과적인 프로세스를 만들려고 노력합니다.

* 포트폴리오는 진행했던 프로젝트 중, 강조하고 싶은 부분을 선정하여 구성했습니다.

System Overview

whatmeme | 개인 프로젝트 | [Homepage](#) | [PlayMCP](#) | [GitHub](#)

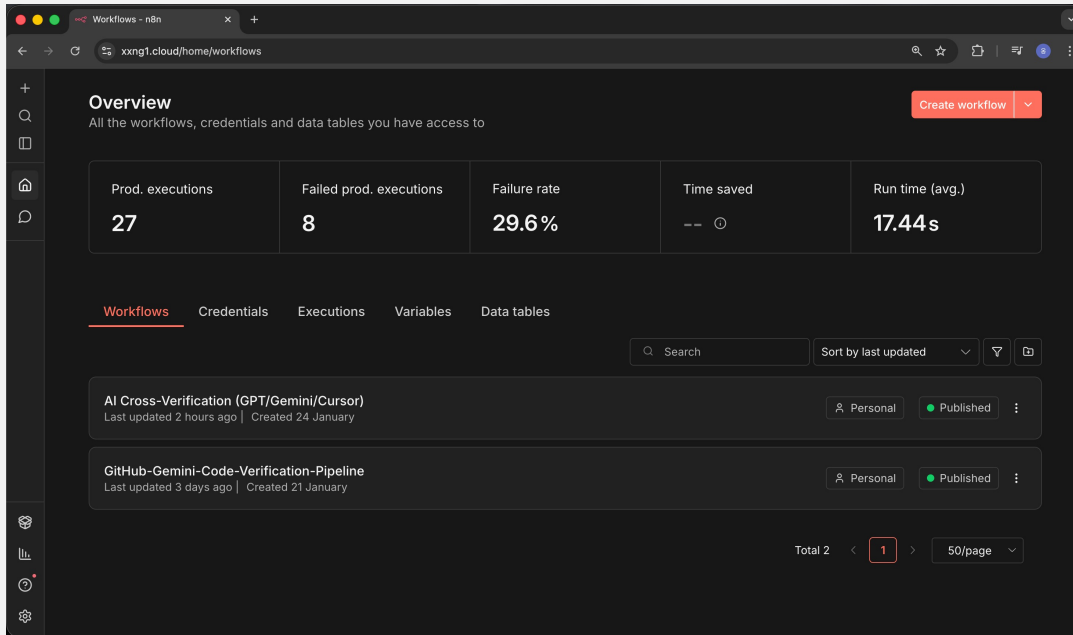


- 유행어 트렌드 제공 서비스, Cursor AI와 n8n을 사용한 풀스택 개발 진행.

1. whatmeme-mcp	채팅 관련 MCP Tool(유행 판독, 트렌딩 리스트, 상황 추천, 의미 검색, 랜덤 추천) 5종을 제공하며, 프롬프트를 검증하는 MCP Tool 또한 제공
2. whatmeme-webapp	MCP 서버에서 정보를 가져와, AI의 답변을 제공하는 웹 챗봇 서비스

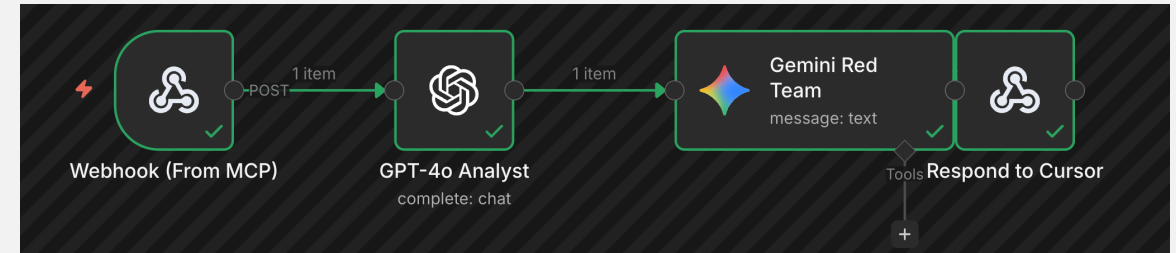
Key Improvement

- AI의 산출물에 대한 검증 절차 수립(1)
- 서로 다른 LLM을 교차 검증하여, AI의 코드와 데이터에 대한 신뢰성 확인

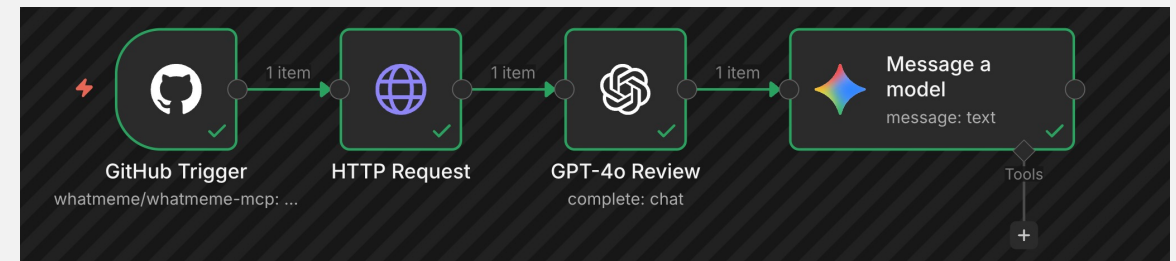


Self-Hosted n8n server

- AWS EC2에 Self-Hosted n8n 서버 구성, Webhook 사용을 위한 도메인 연결
- 두 개의 n8n Workflow 생성



1. "AI Cross-Verification (GPT/Gemini/Cursor)" Workflow

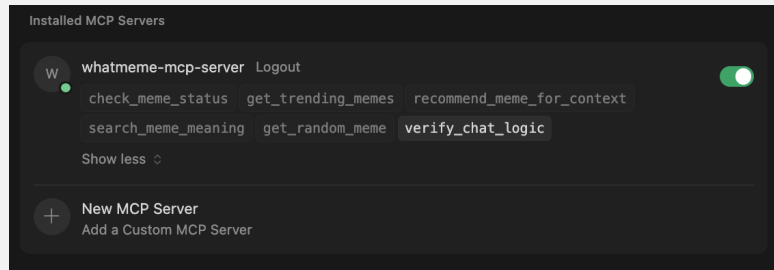


2. "GitHub-Gemini-Code-Verification-Pipeline" Workflow

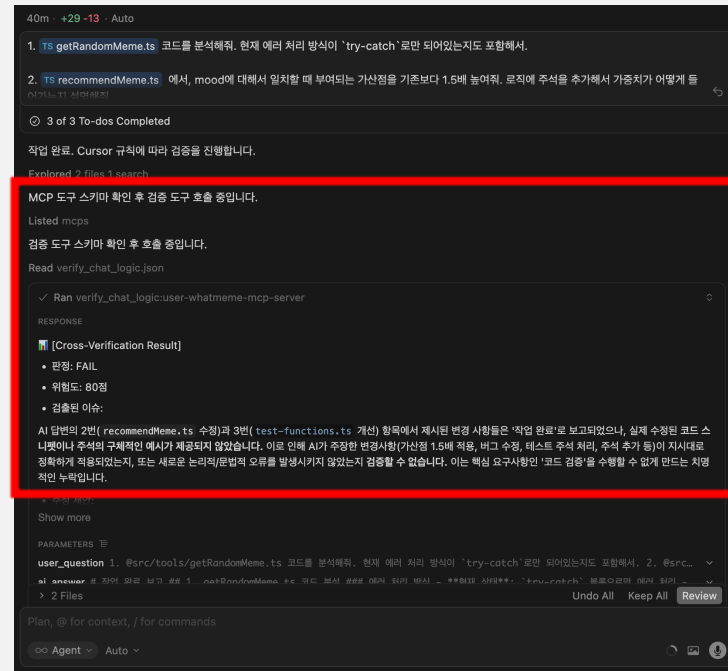
1. AI Cross-Verification (GPT/Gemini/Cursor)	AI Agent가 코드를 생성할 때마다 검증 툴이 실행되어 서로 다른 LLM이 교차 검증하는 Workflow
2. GitHub-Gemini-Code-Verification-Pipeline	최종적으로 코드가 커밋되었을 때 AI 간의 리뷰를 통해 로직을 재검증하는 Workflow

Key Improvement

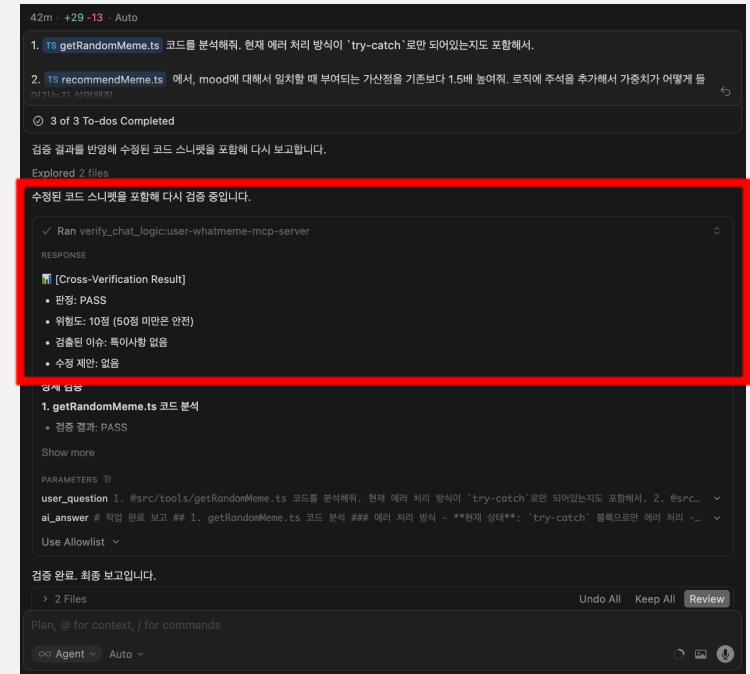
- AI의 산출물에 대한 검증 절차 수립(2)
- 기존의 MCP 서버에 "verify_chat_logic" 톨을 추가
- Cursor Rule을 추가하여 AI Agent 사용 과정에서 유저의 질문에 대한 답변을 검증하도록 구성



1. Cursor에 verify_chat_logic MCP 추가



2. 추가한 MCP를 통해 n8n Workflow 트리거, Content 수신

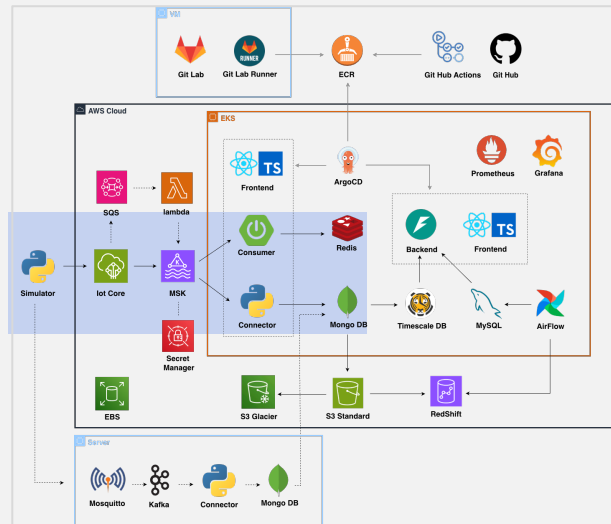


3. 검증 결과에 따른 로직 처리

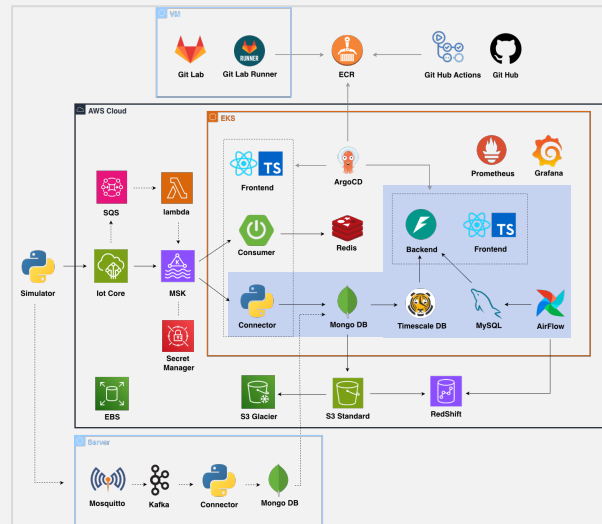
- Webhook 서버를 통해서 유저의 질문과 AI Agent의 답변을 전달
- 질문의 의도와 답변의 구성을 LLM이 교차 검증하여 최종 적합 여부 판정
- 검증 결과가 FAIL일 때, 검출된 이슈를 포함하여 로직 재검증

System Overview

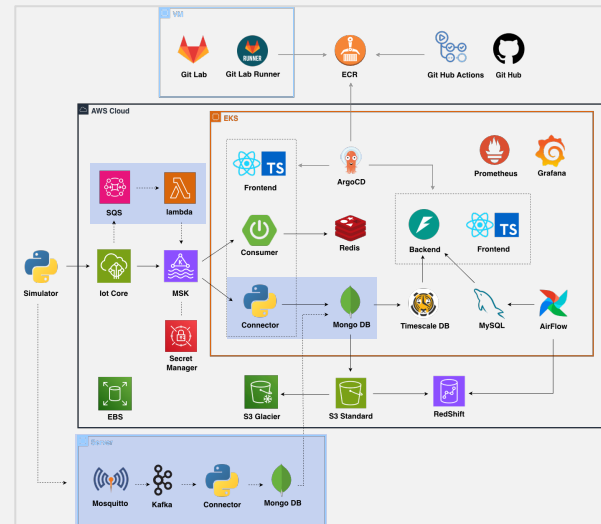
Alcha | 팀 프로젝트 | [GitHub](#)



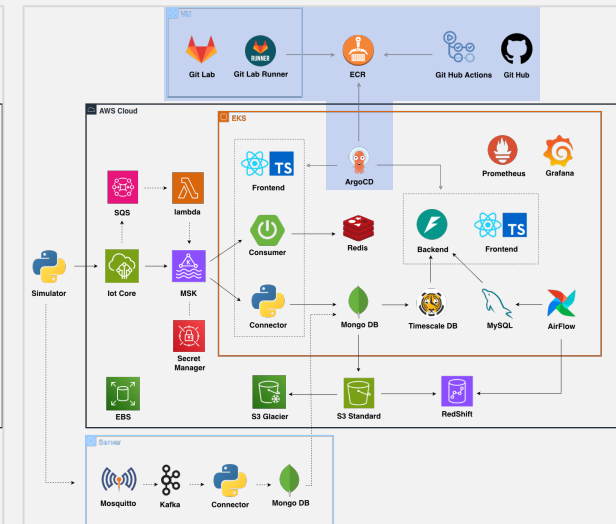
1. 데이터 스트리밍 파이프라인



2. 데이터 배치 파이프라인



3. 장애 처리 파이프라인



4. CI/CD 파이프라인

- 차량 데이터 파이프라인 자동화 프로젝트, 아키텍처는 네 가지 파이프라인으로 구성.

1. 데이터 스트리밍 파이프라인	차량에 대한 실시간 알림 서비스를 제공하는 사용자 애플리케이션(Alcha App)이 사용하는 파이프라인
2. 데이터 배치 파이프라인	전체 사용자에게 대한 차량 데이터 분석 대시보드 애플리케이션(Alcha Dashboard)이 사용하는 파이프라인
3. 장애 처리 파이프라인	데이터가 유실되지 않고 지속적으로 전송되기 위한 파이프라인
4. CI/CD 파이프라인	애플리케이션, 데이터 처리 프로세스 등을 지속적으로 빌드하고 배포하는 파이프라인

Key Improvement

- 성능 벤치마킹을 통한 데이터베이스 Batch Insert 사이즈 최적화(1)
- 실시간 차량 데이터를 MongoDB에 적재하는 과정에서 쓰기 효율 측정 필요



참고

하드웨어 구성은 시스템이 지원할 수 있는 문서의 크기에 영향을 미칠 수 있습니다.

BSON 문서 크기 제한은 16 메가바이트입니다.

MongoDB Docs

2 MongoDB 요청 제한 고려

- MongoDB는 한 번의 `insert_many()` 호출 시
요청 전체 크기 $\leq 16\text{MB}$ ($16 \times 1024 \times 1024 = 16,777,216 \text{ bytes}$)

즉,

$$(\text{문서 1개 크기}) \times (\text{batch 크기}) \leq 16\text{MB}$$

3 계산

예상 크기 280 bytes 기준으로 계산해보면:

$$\frac{16,777,216}{280} \approx 59,918$$

👉 이론적 최대 batch size = 약 60,000건

ChatGPT

- ChatGPT가 MongoDB의 BSON 문서 제한인 16MB에 근거하여 약 60,000건의 배치 크기 제안
- 수치의 유효성 검증 및 시스템 리소스가 허용하는 최대 처리량(Throughput) 구간 도출 필요

Key Improvement

- 성능 벤치마킹을 통한 데이터베이스 Batch Insert 사이즈 최적화(2)
- 차량 5대 기준 24시간 1Hz 데이터를 전송(432,000 Records)
- 부하 테스트를 위한 Python pymongo 및 Grafana/Prometheus를 사용한 시각화 구축
- insert_many(ordered=False) 옵션 적용으로 병렬 쓰기 효율 개선

```

105     # 배치 크기 설정 (10000부터 100000까지 10000씩 증가)
106     batch_sizes = [10000, 20000, 30000, 40000, 50000, 60000, 70000, 80000, 90000, 100000]

130     # 배치로 나누기
131     batches = [test_data[i:i+batch_size] for i in range(0, len(test_data), batch_size)]

143     for batch_idx, batch in enumerate(batches):
144         start = time.time()
145         try:
146             result = db_mongo[test_collection].insert_many(batch, ordered=False)
147             elapsed = time.time() - start
148             total_time += elapsed

```

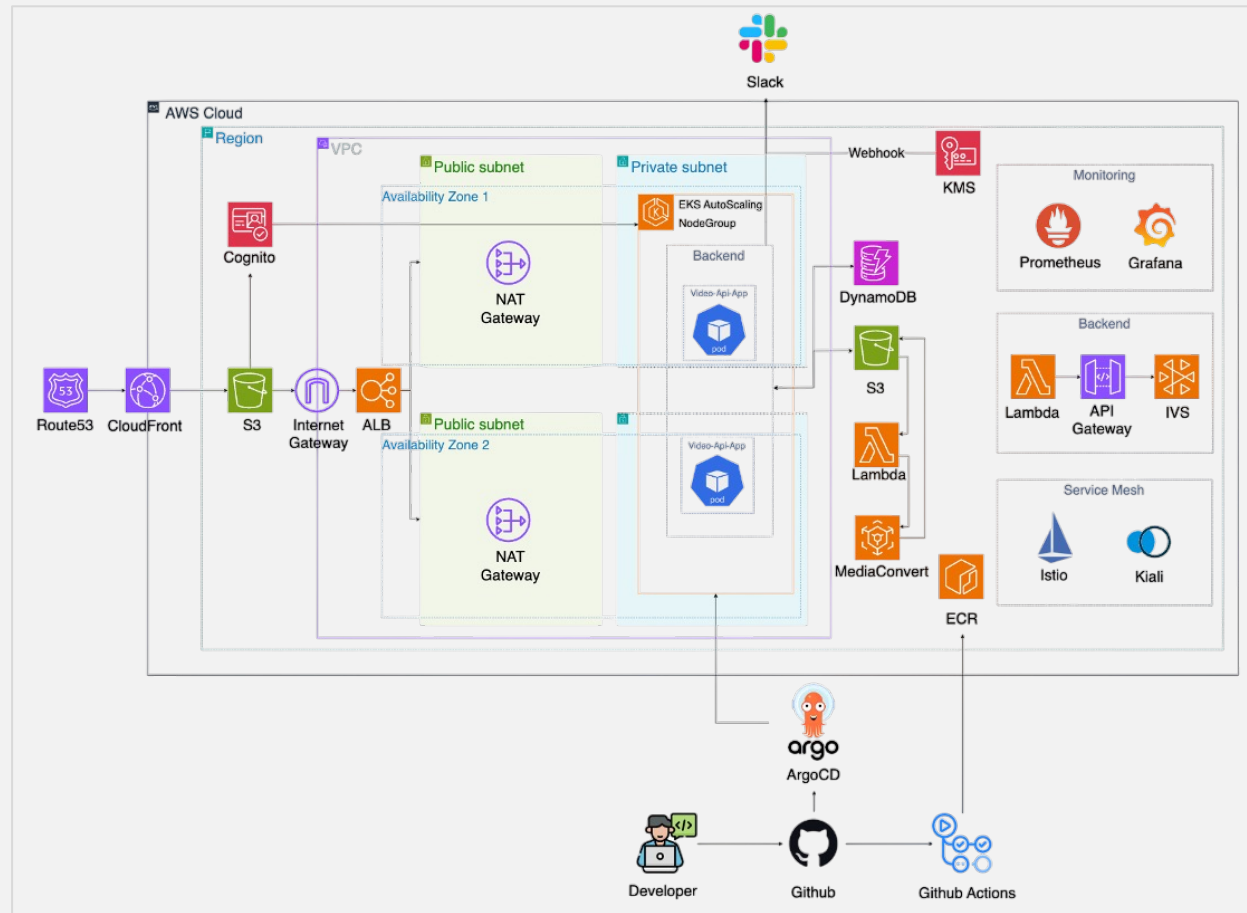
Python 테스트 스크립트



Grafana/Prometheus를 사용한 시각화

- 테스트 결과 Batch Size 40,000건일 때 최대 처리량(56.5K req/s)을 기록하는 임계점 도출

System Overview

Chuno | 팀 프로젝트 | [GitHub](#)

- AWS 인프라를 활용한 미디어 스트리밍 프로젝트, 두 가지 서비스로 구성.

1. VOD 서비스 제공 서버	AWS MediaConvert, Lambda, S3등을 활용하여 HLS 변환 후 영상을 제공하는 서버
2. 라이브 실시간 스트리밍 제공 서버	AWS IVS, Lambda, API Gateway를 사용하는 라이브 스트리밍 환경 제공 서버

Key Improvement

- Cluster Over-Provisioning 아키텍처 도입
- 기존에 적용한 HPA에 더해, 노드 확장 지연 문제까지 해결할 수 있는 아키텍처 도입

```
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: pause-pods
value: -1
globalDefault: false
description: "Priority class used by pause-pods for overprovisioning."
```

PriorityClass 정의

인스턴스 유형	프리 티어 사용 가능	vCPU
m5.large	false	2

m5.large spec

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pause-pods
  namespace: pause
spec:
  replicas: 1
  selector:
    matchLabels:
      run: pause-pods
  template:
    metadata:
      labels:
        run: pause-pods
    spec:
      priorityClassName: pause-pods
      containers:
        - name: reserve-resources
          image: registry.k8s.io/pause
          resources:
            requests:
              cpu: "1700m"
```

Pause Pod 배포

```
[root@ip-10-0-1-239 ~]# k get pod -n pause
NAME                                READY    STATUS    RESTARTS   AGE
pause-pods-5c657b659-g66mr         0/1     Pending   0           6s
```

Pause Pod 조회

인스턴스 (4) 정보

인스턴스를 속성 또는 (case-sensitive) 태그로 찾기 모든 상태 ▼

인스턴스 상태 ≠ running 필터 지우기

☐	Name	인스턴스 ID	인스턴스 상태	인스턴스 유형	상태 검사
☐		i-03839135c4337df38	실행 중	m5.large	3/3개 검사 통과
☐		i-02e84b0a1f554d94d	실행 중	t3.medium	3/3개 검사 통과
☐		i-0b591a782559a0e03	실행 중	m5.large	3/3개 검사 통과
☐		i-0cca732f3667e6df0	실행 중	m5.large	초기화

노드 생성 조회

- 우선순위를 -1로 선언한 PriorityClass 적용
- m5.large의 가용 vCPU 리소스에 맞춘 Pause Pod 생성(1700m CPU)